HƯỚNG DẪN CHI TIẾT GIẢI ĐỀ PRM393 – REVISION 1: FRUIT SALES MANAGEMENT

(Dựa trên cấu trúc chuẩn của project `sqlite_intro` — Đầy đủ 100 điểm từ Task 1 đến Task 6 & khớp hoàn toàn với Illustrations a → f)

1. Lưu Ý Sống Còn Khi Làm Đề Này (Tránh Bị 0 Điểm)

Ở trang 4 của đề thi có quy định rất rõ:

> **Rules**: *"The project will receive zero points if it fails to run, or if it contains data structures, variables, images, screens, or UI elements unrelated to the examination topic."*

Vì vậy:

1. **Đặt tên project đúng chuẩn**: `fa26_prm393_<StudentCode>` (ví dụ: `fa26_prm393_se190321`).
2. **Xóa sạch các file thừa**: Không để lại file `note.dart`, `note_list_screen.dart` hay các biến/màn hình của bài khác. Chỉ giữ đúng các file phục vụ bài **Fruit Sales Management**.
3. **Trước khi nộp bài**: Chạy lệnh `flutter clean` trong Terminal theo đúng yêu cầu đề.

2. Phân Tích Công Thức Tính Ngày Hết Hạn (`daysLeft`) Khớp 100% Bảng Sample Data

Ở **Task 2, Task 3, Task 5**, đề bài yêu cầu tính số ngày còn lại: `daysLeft = expiredDate.difference(today).inDays` và phân loại thành 3 nhóm:

- **`Expired`**: `daysLeft < 0`
- **`Expiring soon`**: `0 <= daysLeft <= 7`
- **`Fresh`**: `daysLeft > 7`

> IMPORTANT
>
> **Bẫy cực dễ mất điểm trong Dart**: Nếu bạn lấy `DateTime.now()` trực tiếp (có chứa giờ, phút, giây hiện tại, ví dụ `2026-11-08 14:30:00`) mà lấy `expiredDate` (`2026-11-09 00:00:00`) trừ

> đi, hiệu số chỉ là `9 tiếng 30 phút`, khi gọi `.inDays` sẽ bị làm tròn xuống thành **`0 days`** thay vì **`1 day`**!
>
> **Cách viết chuẩn xác 100%**: Luôn đưa cả `today` và `expiredDate` về mốc `00:00:00` (`DateTime(year, month, day)`) trước khi gọi `.difference(today).inDays`:
>
> ```
> dart
> static int calculateDaysLeft(String expiredDateStr) {
>   final now = DateTime.now();
>   final today = DateTime(now.year, now.month, now.day);
>   final parsed = DateTime.parse(expiredDateStr); // YYYY-MM-DD
>   final expiry = DateTime(parsed.year, parsed.month, parsed.day);
>   return expiry.difference(today).inDays;
> }
> ```
>
> Với hàm này, khi kiểm thử ở ngày `2026-11-08`, hạn `2026-11-09` ra đúng **`1 day`**, hạn `2026-11-08` ra đúng **`0 day`**, hạn `2026-11-15` ra đúng **`7 days`** — khớp từng con số với bảng **Sample Data** ở trang 2 của đề!

3. Cấu Trúc Thư Mục Project (Theo Đúng Tư Duy `sqlite_intro`)

```
text
lib/
├── main.dart                        # Khởi tạo ProviderScope & MaterialApp
├── model/
│   └── fruit.dart                   # Model Fruit (id, name, origin, price,
expiredDate + hàm tính daysLeft)
├── db/
│   └── database_helper.dart         # Singleton quản lý bảng 'fruits' (getAll,
insert, updateExpiryDate, delete)
├── providers/
│   └── fruit_notifier.dart          # Quản lý state danh sách Fruit toàn cục
└── ui/
    ├── fruit_list_screen.dart       # Task 1 & Task 4: Danh sách + Empty State +
Delete Dialog (Hình a, b, e)
    ├── add_fruit_screen.dart        # Task 2: Form thêm mới + DatePicker + Dynamic
Badge (Hình c)
    ├── update_expiry_screen.dart    # Task 3: Form chỉ sửa ngày hết hạn + Dynamic
Badge (Hình d)
    └── expiry_warning_screen.dart   # Task 5: Màn hình cảnh báo Expiring soon &
Expired (Hình f)
```

4. Hướng Dẫn Code Chi Tiết Từng Bước

Bước 1: Tạo Model `lib/model/fruit.dart` (Data Structure + Logic `daysLeft`)

Giải thích

- Khai báo đúng 5 trường trong bảng `fruits`: `id`, `name`, `origin`, `price`, `expiredDate`.
- Viết hàm `toJson()` và `factory Fruit.fromJson(...)` giống hệt `note.dart` của `sqlite_intro`.
- Tích hợp sẵn các helper tính `daysLeft` và nhãn trạng thái (`Fresh`, `Expiring soon`, `Expired`) để tái sử dụng ở cả 4 màn hình.

Code `lib/model/fruit.dart`

```
dart
class Fruit {
  final int? id;
  final String name;
  final String origin;
  final double price;
  final String expiredDate; // Định dạng YYYY-MM-DD

  Fruit({
    this.id,
    required this.name,
    required this.origin,
    required this.price,
    required this.expiredDate,
  });

  // Chuyển Object thành Map để lưu vào SQLite
  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id,
      'name': name,
      'origin': origin,
      'price': price,
      'expiredDate': expiredDate,
    };
  }

  // Đọc Map từ SQLite thành Object Fruit
  factory Fruit.fromJson(Map<String, dynamic> json) {
    return Fruit(
      id: json['id'] as int?,
      name: json['name'] as String? ?? '',
      origin: json['origin'] as String? ?? '',
      price: (json['price'] as num?)?.toDouble() ?? 0.0,
      expiredDate: json['expiredDate'] as String? ?? '',
    );
  }

  // Tính số ngày còn lại: daysLeft = expiredDate.difference(today).inDays
  static int computeDaysLeft(String isoDate) {
    final parsed = DateTime.tryParse(isoDate);
    if (parsed == null) return 0;

    final now = DateTime.now();
```

```
    final today = DateTime(now.year, now.month, now.day);
    final expiry = DateTime(parsed.year, parsed.month, parsed.day);
    return expiry.difference(today).inDays;
  }

  int get daysLeft => computeDaysLeft(expiredDate);

  // Phân nhóm: Expired (< 0), Expiring soon (0..7), Fresh (> 7)
  static String statusLabelForDate(String isoDate) {
    final days = computeDaysLeft(isoDate);
    if (days < 0) return 'Expired';
    if (days <= 7) return 'Expiring soon';
    return 'Fresh';
  }

  String get statusLabel => statusLabelForDate(expiredDate);
}
```

Bước 2: Viết `lib/db/database_helper.dart` (SQLite Database)

Giải thích

- Giữ nguyên cấu trúc Singleton `DatabaseHelper.instance` của `sqlite_intro`.
- Tạo bảng `fruits` với đúng tên cột và kiểu dữ liệu đề bài yêu cầu:
  - `id INTEGER PRIMARY KEY AUTOINCREMENT`
  - `name TEXT NOT NULL`
  - `origin TEXT`
  - `price REAL NOT NULL`
  - `expiredDate TEXT NOT NULL`
- Ở **Task 3 (Update expiry date)**, đề yêu cầu: *"After the user taps 'Update', the new expiry date is saved to the SQLite database and the List Screen is refreshed. Other fields are not edited."* Do đó ta viết hàm `updateExpiryDate(int id, String newExpiredDate)` chỉ cập nhật đúng cột `expiredDate`.

Code `lib/db/database_helper.dart`

```
dart
import 'package:flutter/foundation.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

import '../model/fruit.dart';

class DatabaseHelper {
  static const _databaseName = 'fruit_sales_management.db';
  static const _fruitTableName = 'fruits';

  // Singleton pattern chuẩn sqlite_intro
  DatabaseHelper._privateConstructor();
  static final DatabaseHelper instance = DatabaseHelper._privateConstructor();

  static Database? _database;
  Future<Database> get database async {
    _database ??= await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    if (kIsWeb) {
      databaseFactory = databaseFactoryFfiWeb;
      return openDatabase(_databaseName, version: 1, onCreate: _onCreate);
    }
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, _databaseName);
    return openDatabase(path, version: 1, onCreate: _onCreate);
  }

  Future<void> _onCreate(Database db, int version) async {
    await db.execute('''
      CREATE TABLE $_fruitTableName (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        origin TEXT,
        price REAL NOT NULL,
        expiredDate TEXT NOT NULL
      )
    ''');
  }

  // Task 1: Lấy toàn bộ danh sách trái cây
```

```
  Future<List<Fruit>> getAll() async {
    final db = await database;
    final List<Map<String, Object?>> maps = await db.query(
      _fruitTableName,
      orderBy: 'id ASC',
    );
    return [for (final map in maps) Fruit.fromJson(map)];
  }

  // Task 2: Thêm mới trái cây
  Future<int> insert(Fruit fruit) async {
    final db = await database;
    return db.insert(_fruitTableName, fruit.toJson());
  }

  // Task 3: Chỉ cập nhật ngày hết hạn (expiredDate) của trái cây
  Future<int> updateExpiryDate(int id, String newExpiredDate) async {
    final db = await database;
    return db.update(
      _fruitTableName,
      {'expiredDate': newExpiredDate},
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  // Task 4: Xóa trái cây theo id
  Future<int> delete(int id) async {
    final db = await database;
    return db.delete(
      _fruitTableName,
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}
```

Bước 3: Viết State Management `lib/providers/fruit_notifier.dart`

Giải thích

- Giống hệt `note_notifier.dart` trong `sqlite_intro`, mỗi khi gọi `addFruit`, `updateFruitExpiry`, hoặc `deleteFruit`, ta gọi `ref.invalidateSelf(); await future;` để

toàn bộ các màn hình đang lắng nghe (`FruitListScreen` và `ExpiryWarningScreen`) tự động tính toán lại và cập nhật giao diện ngay lập tức.

Code `lib/providers/fruit_notifier.dart`

```
dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/fruit.dart';

class FruitNotifier extends AsyncNotifier<List<Fruit>> {
  @override
  Future<List<Fruit>> build() async {
    return DatabaseHelper.instance.getAll();
  }

  // Task 2: Thêm trái cây mới
  Future<void> addFruit(Fruit fruit) async {
    await DatabaseHelper.instance.insert(fruit);
    ref.invalidateSelf();
    await future;
  }

  // Task 3: Cập nhật ngày hết hạn
  Future<void> updateFruitExpiry(int id, String newExpiredDate) async {
    await DatabaseHelper.instance.updateExpiryDate(id, newExpiredDate);
    ref.invalidateSelf();
    await future;
  }

  // Task 4: Xóa trái cây
  Future<void> deleteFruit(int id) async {
    await DatabaseHelper.instance.delete(id);
    ref.invalidateSelf();
    await future;
  }
}

final fruitProvider = AsyncNotifierProvider<FruitNotifier, List<Fruit>>(
  FruitNotifier.new,
);
```

Bước 4: Tạo Màn Hình Danh Sách Trái Cây `lib/ui/fruit_list_screen.dart` (Task 1 & Task 4 — Hình a, b, e)

Giải thích

1. `AppBar`:
    - Hiển thị tiêu đề `"Fruit Shop"` và nút action `Icons.event_busy` (hoặc `Icons.warning_amber_rounded`) để mở màn hình `ExpiryWarningScreen` (Task 5).
2. **Khi danh sách rỗng (Hình b)**:
    - Hiển thị icon chiếc lá `Icons.eco` và dòng chữ `"No fruits yet"` như hình minh họa (b).
3. **Khi có dữ liệu (Hình a)**:
    - Hiển thị bằng `ListView.separated` (hoặc `ListView.builder`), mỗi phần tử là một `ListTile`:
        - `leading`: `Icons.eco` đặt trong `CircleAvatar`.
        - `title`: `fruit.name`.
        - `subtitle`: `"Origin – Price – Expiry date"` (kèm nhãn nhỏ `"Soon"` nếu `0 <= daysLeft <= 7` hoặc `"Expired"` nếu `daysLeft < 0` giống hệt Hình a).
        - `trailing`: 2 nút `IconButton` gồm `Icons.edit` (mở `UpdateExpiryScreen` - Task 3) và `Icons.delete` (mở `AlertDialog` xác nhận xóa - Task 4).
4. **Hộp thoại xác nhận xóa (Task 4 — Hình e)**:
    - Hiển thị tên trái cây được chọn: `'Are you sure you want to delete "${fruit.name}"? This action cannot be undone.'` với 2 nút `Cancel` và `Delete`.

Code `lib/ui/fruit_list_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/fruit.dart';
import '../providers/fruit_notifier.dart';
import 'add_fruit_screen.dart';
import 'expiry_warning_screen.dart';
import 'update_expiry_screen.dart';

class FruitListScreen extends ConsumerWidget {
  const FruitListScreen({super.key});

  // Định dạng giá tiền có dấu phẩy ngăn cách hàng nghìn: 25000 -> "25,000đ"
  static String formatPrice(double price) {
    final str = price.round().toString();
    final buffer = StringBuffer();
    for (int i = 0; i < str.length; i++) {
      if (i > 0 && (str.length - i) % 3 == 0) {
        buffer.write(',');
      }
      buffer.write(str[i]);
    }
    return '${buffer.toString()}đ';
  }

  // Task 4: Hộp thoại xác nhận xóa hiển thị tên trái cây (Hình e)
  Future<void> _confirmDelete(
    BuildContext context,
    WidgetRef ref,
    Fruit fruit,
  ) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Delete fruit?'),
        content: Text(
          'Are you sure you want to delete "${fruit.name}"? This action cannot be
undone.',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Cancel'),
          ),
```

```
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.red.shade700,
              foregroundColor: Colors.white,
            ),
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Delete'),
          ),
        ],
      ),
    );

    if (confirmed == true) {
      await ref.read(fruitProvider.notifier).deleteFruit(fruit.id!);
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final fruitsAsync = ref.watch(fruitProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Fruit Shop',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
            ),
            Text(
              'Fresh fruits & expiry tracking',
              style: TextStyle(fontSize: 12, color: Colors.white70),
            ),
          ],
        ),
        actions: [
          // Task 5: Nút mở màn hình Expiry Warning Screen
          IconButton(
            tooltip: 'Expiry Warnings',
            icon: const Icon(Icons.event_busy),
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (_) => const ExpiryWarningScreen()),
              );
```

```
            },
          ),
        ],
      ),
      body: fruitsAsync.when(
        data: (fruits) => fruits.isEmpty
            // Hình (b): Empty state khi danh sách trống
            ? Center(
                child: Padding(
                  padding: const EdgeInsets.all(24.0),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      CircleAvatar(
                        radius: 36,
                        backgroundColor: Colors.green.shade50,
                        child: Icon(
                          Icons.eco,
                          size: 36,
                          color: Colors.green.shade700,
                        ),
                      ),
                      const SizedBox(height: 16),
                      const Text(
                        'No fruits yet',
                        style: TextStyle(
                          fontSize: 18,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      const SizedBox(height: 8),
                      const Text(
                        'Tap the + button to add your first fruit and start
tracking expiry dates.',
                        textAlign: TextAlign.center,
                        style: TextStyle(color: Colors.black54),
                      ),
                    ],
                  ),
                ),
              )
            // Hình (a): Danh sách trái cây sử dụng ListView và ListTile
            : ListView.separated(
                padding: const EdgeInsets.symmetric(vertical: 8),
                itemCount: fruits.length,
                separatorBuilder: (_, __) => const Divider(height: 1),
```

```
                itemBuilder: (context, index) {
                  final fruit = fruits[index];
                  final days = fruit.daysLeft;
                  final originText =
                      fruit.origin.trim().isEmpty ? 'N/A' : fruit.origin;

                  return ListTile(
                    leading: CircleAvatar(
                      backgroundColor: Colors.green.shade100,
                      child: Icon(Icons.eco, color: Colors.green.shade800),
                    ),
                    title: Text(
                      fruit.name,
                      style: const TextStyle(fontWeight: FontWeight.bold),
                    ),
                    // Subtitle: "Origin — Price — Expiry date" kèm nhãn trạng thái
                    subtitle: Wrap(
                      crossAxisAlignment: WrapCrossAlignment.center,
                      spacing: 6,
                      children: [
                        Text(
                          '$originText · ${formatPrice(fruit.price)} · Exp
${fruit.expiredDate}',
                          style: const TextStyle(fontSize: 12.5),
                        ),
                        if (days < 0)
                          _buildMiniTag('Expired', Colors.red.shade100,
Colors.red.shade800)
                        else if (days <= 7)
                          _buildMiniTag('Soon', Colors.orange.shade100,
Colors.orange.shade900),
                      ],
                    ),
                    // Trailing: Icons.edit (Task 3) và Icons.delete (Task 4)
                    trailing: Row(
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        IconButton(
                          icon: const Icon(Icons.edit, size: 20),
                          onPressed: () {
                            Navigator.push(
                              context,
                              MaterialPageRoute(
                                builder: (_) => UpdateExpiryScreen(fruit: fruit),
                              ),
                            );
```

```
                        },
                      ),
                      IconButton(
                        icon: const Icon(Icons.delete, size: 20),
                        onPressed: () => _confirmDelete(context, ref, fruit),
                      ),
                    ],
                  ),
                );
              },
            ),
        error: (err, _) => Center(child: Text('Error loading fruits: $err')),
        loading: () => const Center(child: CircularProgressIndicator()),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const AddFruitScreen()),
          );
        },
        child: const Icon(Icons.add),
      ),
    );
  }

  Widget _buildMiniTag(String text, Color bg, Color fg) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(4),
      ),
      child: Text(
        text,
        style: TextStyle(fontSize: 10, fontWeight: FontWeight.bold, color: fg),
      ),
    );
  }
}
```

Bước 5: Tạo Màn Hình Thêm Trái Cây `lib/ui/add_fruit_screen.dart` (Task 2 — Hình c)

Giải thích

- Gồm 4 trường trong `Form`: `Name *`, `Origin`, `Price (VND/kg) *`, `Expiry date *`.
- Kiểm tra dữ liệu bắt buộc bằng `validator` trước khi lưu.
- Ô `Price` sử dụng bàn phím số (`keyboardType: TextInputType.number`).
- Ô `Expiry date` mở `showDatePicker` và lưu chuỗi định dạng `YYYY-MM-DD`.
- **Dynamic Status Badge**: Ngay bên dưới ô chọn ngày, hiển thị badge trạng thái (`Fresh` màu xanh lá / `Expiring soon` màu cam / `Expired` màu đỏ) kèm số ngày còn lại (`X days left` / `X days ago`), tự động cập nhật ngay khi người dùng chọn ngày mới!
- Nhấn `"Save"` -> Lưu vào SQLite -> Quay về `FruitListScreen` và danh sách tự động cập nhật.
- Nhớ `dispose()` toàn bộ các `TextEditingController` trong hàm `dispose()` để đạt trọn điểm **Task 6**.

Code `lib/ui/add_fruit_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/fruit.dart';
import '../providers/fruit_notifier.dart';

class AddFruitScreen extends ConsumerStatefulWidget {
  const AddFruitScreen({super.key});

  @override
  ConsumerState<AddFruitScreen> createState() => _AddFruitScreenState();
}

class _AddFruitScreenState extends ConsumerState<AddFruitScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameCtrl = TextEditingController();
  final _originCtrl = TextEditingController();
  final _priceCtrl = TextEditingController();
  final _expiryCtrl = TextEditingController();

  @override
  void initState() {
    super.initState();
    // Khởi tạo ngày mặc định là hôm nay theo chuẩn YYYY-MM-DD
    final now = DateTime.now();
    _expiryCtrl.text = _formatIsoDate(now);
  }

  @override
  void dispose() {
    // Task 6: Dispose the owned controllers after use
    _nameCtrl.dispose();
    _originCtrl.dispose();
    _priceCtrl.dispose();
    _expiryCtrl.dispose();
    super.dispose();
  }

  String _formatIsoDate(DateTime date) {
    final y = date.year.toString().padLeft(4, '0');
    final m = date.month.toString().padLeft(2, '0');
    final d = date.day.toString().padLeft(2, '0');
    return '$y-$m-$d';
  }
```

```
  Future<void> _pickExpiryDate() async {
    final initial = DateTime.tryParse(_expiryCtrl.text) ?? DateTime.now();
    final picked = await showDatePicker(
      context: context,
      initialDate: initial,
      firstDate: DateTime(2020),
      lastDate: DateTime(2035),
    );
    if (picked != null) {
      setState(() {
        _expiryCtrl.text = _formatIsoDate(picked);
      });
    }
  }

  Future<void> _saveFruit() async {
    if (!_formKey.currentState!.validate()) return;

    final newFruit = Fruit(
      name: _nameCtrl.text.trim(),
      origin: _originCtrl.text.trim(),
      price: double.parse(_priceCtrl.text.trim()),
      expiredDate: _expiryCtrl.text.trim(),
    );

    await ref.read(fruitProvider.notifier).addFruit(newFruit);
    if (mounted) {
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Add Fruit',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
            ),
            Text(
              'Create a new fruit',
              style: TextStyle(fontSize: 12, color: Colors.white70),
            ),
          ],
```

```
      ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // 1. Name
            TextFormField(
              controller: _nameCtrl,
              decoration: const InputDecoration(
                labelText: 'Name *',
                border: OutlineInputBorder(),
              ),
              validator: (val) {
                if (val == null || val.trim().isEmpty) {
                  return 'Name is required';
                }
                return null;
              },
            ),
            const SizedBox(height: 16),

            // 2. Origin
            TextFormField(
              controller: _originCtrl,
              decoration: const InputDecoration(
                labelText: 'Origin',
                border: OutlineInputBorder(),
              ),
              validator: (val) {
                if (val == null || val.trim().isEmpty) {
                  return 'Origin is required';
                }
                return null;
              },
            ),
            const SizedBox(height: 16),

            // 3. Price (Number keyboard type)
            TextFormField(
              controller: _priceCtrl,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
```

```
                  labelText: 'Price (VND/kg) *',
                  border: OutlineInputBorder(),
                ),
                validator: (val) {
                  if (val == null || val.trim().isEmpty) {
                    return 'Price is required';
                  }
                  final number = double.tryParse(val.trim());
                  if (number == null || number <= 0) {
                    return 'Please enter a valid price > 0';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 16),

              // 4. Expiry Date (Chosen with showDatePicker and stored as YYYY-MM-
DD)
              TextFormField(
                controller: _expiryCtrl,
                readOnly: true,
                onTap: _pickExpiryDate,
                decoration: InputDecoration(
                  labelText: 'Expiry date *',
                  border: const OutlineInputBorder(),
                  suffixIcon: IconButton(
                    icon: const Icon(Icons.calendar_today),
                    onPressed: _pickExpiryDate,
                  ),
                ),
                validator: (val) {
                  if (val == null || val.trim().isEmpty) {
                    return 'Expiry date is required';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 10),

              // Dynamic Status Badge (Fresh / Expiring soon / Expired)
              ExpiryStatusBadge(isoDate: _expiryCtrl.text),
              const SizedBox(height: 6),
              const Text(
                'The expiry date is chosen with a date picker and stored as YYYY-
MM-DD.',
                style: TextStyle(fontSize: 12, color: Colors.black54),
```

```
              ),
              const SizedBox(height: 24),

              // Save Button
              SizedBox(
                width: double.infinity,
                height: 48,
                child: ElevatedButton(
                  onPressed: _saveFruit,
                  child: const Text(
                    'Save',
                    style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

/// Widget hiển thị Status Badge động (Fresh / Expiring soon / Expired) dùng chung cho Task 2 & Task 3
class ExpiryStatusBadge extends StatelessWidget {
  final String isoDate;

  const ExpiryStatusBadge({super.key, required this.isoDate});

  @override
  Widget build(BuildContext context) {
    final days = Fruit.computeDaysLeft(isoDate);
    final status = Fruit.statusLabelForDate(isoDate);

    Color bgColor;
    Color textColor;
    String detailText;

    if (days < 0) {
      bgColor = Colors.red.shade100;
      textColor = Colors.red.shade800;
      detailText = '${days.abs()} day${days.abs() == 1 ? '' : 's'} ago';
    } else if (days <= 7) {
      bgColor = Colors.orange.shade100;
      textColor = Colors.orange.shade900;
      detailText = '$days day${days == 1 ? '' : 's'} left';
```

```
    } else {
      bgColor = Colors.green.shade100;
      textColor = Colors.green.shade800;
      detailText = '$days days left';
    }

    return Row(
      children: [
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
          decoration: BoxDecoration(
            color: bgColor,
            borderRadius: BorderRadius.circular(6),
          ),
          child: Text(
            status,
            style: TextStyle(
              color: textColor,
              fontWeight: FontWeight.bold,
              fontSize: 12,
            ),
          ),
        ),
        const SizedBox(width: 8),
        Text(
          detailText,
          style: const TextStyle(fontSize: 12.5, color: Colors.black54),
        ),
      ],
    );
  }
}
```

Bước 6: Tạo Màn Hình Cập Nhật Ngày Hết Hạn `lib/ui/update_expiry_screen.dart` (Task 3 — Hình d)

Giải thích

- Đề bài yêu cầu:
  - Điền sẵn ngày hết hạn hiện tại của trái cây được chọn (`widget.fruit.expiredDate`).
  - Cho phép chọn ngày mới bằng `showDatePicker` .
  - Badge trạng thái (`Fresh` / `Expiring soon` / `Expired`) cập nhật động khi đổi ngày.

- Các trường khác (`name` , `origin` , `price`) chỉ hiển thị trong khung thông tin phía trên, **không chỉnh sửa**.
- Bấm `"Update"` -> Gọi `updateFruitExpiry` lưu xuống SQLite và quay về làm mới `FruitListScreen` .

Code `lib/ui/update_expiry_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/fruit.dart';
import '../providers/fruit_notifier.dart';
import 'add_fruit_screen.dart';
import 'fruit_list_screen.dart';

class UpdateExpiryScreen extends ConsumerStatefulWidget {
  final Fruit fruit;

  const UpdateExpiryScreen({super.key, required this.fruit});

  @override
  ConsumerState<UpdateExpiryScreen> createState() => _UpdateExpiryScreenState();
}

class _UpdateExpiryScreenState extends ConsumerState<UpdateExpiryScreen> {
  late final TextEditingController _expiryCtrl;

  @override
  void initState() {
    super.initState();
    // Pre-fill với ngày hết hạn hiện tại của trái cây được chọn
    _expiryCtrl = TextEditingController(text: widget.fruit.expiredDate);
  }

  @override
  void dispose() {
    _expiryCtrl.dispose();
    super.dispose();
  }

  String _formatIsoDate(DateTime date) {
    final y = date.year.toString().padLeft(4, '0');
    final m = date.month.toString().padLeft(2, '0');
    final d = date.day.toString().padLeft(2, '0');
    return '$y-$m-$d';
  }

  Future<void> _pickNewDate() async {
    final current = DateTime.tryParse(_expiryCtrl.text) ?? DateTime.now();
    final picked = await showDatePicker(
      context: context,
      initialDate: current,
```

```
      firstDate: DateTime(2020),
      lastDate: DateTime(2035),
    );
    if (picked != null) {
      setState(() {
        _expiryCtrl.text = _formatIsoDate(picked);
      });
    }
  }

  Future<void> _updateExpiry() async {
    if (_expiryCtrl.text.trim().isEmpty) return;
    await ref
        .read(fruitProvider.notifier)
        .updateFruitExpiry(widget.fruit.id!, _expiryCtrl.text.trim());
    if (mounted) {
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    final fruit = widget.fruit;
    final originText = fruit.origin.trim().isEmpty ? 'N/A' : fruit.origin;

    return Scaffold(
      appBar: AppBar(
        title: const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Update Expiry',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
            ),
            Text(
              'Edit expiry date only',
              style: TextStyle(fontSize: 12, color: Colors.white70),
            ),
          ],
        ),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
```

```
// Khung hiển thị thông tin trái cây (Không chỉnh sửa – Hình d)
Container(
  width: double.infinity,
  padding: const EdgeInsets.all(14),
  decoration: BoxDecoration(
    color: Colors.green.shade50,
    borderRadius: BorderRadius.circular(8),
    border: Border.all(color: Colors.green.shade200),
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        'Fruit',
        style: TextStyle(fontSize: 12, color: Colors.green.shade800),
      ),
      const SizedBox(height: 4),
      Text(
        fruit.name,
        style: const TextStyle(
          fontSize: 17,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 2),
      Text(
        '$originText · ${FruitListScreen.formatPrice(fruit.price)}',
        style: const TextStyle(color: Colors.black54, fontSize: 13),
      ),
    ],
  ),
),
const SizedBox(height: 20),

// Ô chọn ngày hết hạn mới
TextFormField(
  controller: _expiryCtrl,
  readOnly: true,
  onTap: _pickNewDate,
  decoration: InputDecoration(
    labelText: 'Expiry date *',
    border: const OutlineInputBorder(),
    suffixIcon: IconButton(
      icon: const Icon(Icons.calendar_today),
      onPressed: _pickNewDate,
    ),
```

```
              ),
            ),
            const SizedBox(height: 10),

            // Dynamic Status Badge
            ExpiryStatusBadge(isoDate: _expiryCtrl.text),
            const SizedBox(height: 6),
            const Text(
              'Only the expiry date can be updated. The name, origin and price are
not editable here.',
              style: TextStyle(fontSize: 12, color: Colors.black54),
            ),
            const SizedBox(height: 24),

            // Nút Update
            SizedBox(
              width: double.infinity,
              height: 48,
              child: ElevatedButton(
                onPressed: _updateExpiry,
                child: const Text(
                  'Update',
                  style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bước 7: Tạo Màn Hình Cảnh Báo Hết Hạn `lib/ui/expiry_warning_screen.dart` (Task 5 — Hình f)

Giải thích

- Tính `daysLeft = expiredDate.difference(today).inDays` cho từng trái cây.
- Chia làm 2 khu vực rõ ràng như Hình (f):
    1. **`EXPIRING SOON`** (`0 <= daysLeft <= 7`) kèm con số đếm số lượng trái cây ở góc phải tiêu đề.

2. **`EXPIRED`** (`daysLeft < 0`) kèm con số đếm số lượng trái cây ở góc phải tiêu đề.

- Vì màn hình này dùng `ref.watch(fruitProvider)`, bất cứ khi nào thêm mới hoặc cập nhật hạn sử dụng, danh sách cảnh báo luôn được tính toán lại và làm mới tự động.

Code `lib/ui/expiry_warning_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/fruit.dart';
import '../providers/fruit_notifier.dart';

class ExpiryWarningScreen extends ConsumerWidget {
  const ExpiryWarningScreen({super.key});

  String _todayIso() {
    final now = DateTime.now();
    final y = now.year.toString().padLeft(4, '0');
    final m = now.month.toString().padLeft(2, '0');
    final d = now.day.toString().padLeft(2, '0');
    return '$y-$m-$d';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final fruitsAsync = ref.watch(fruitProvider);

    return Scaffold(
      appBar: AppBar(
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Expiry Warnings',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
            ),
            Text(
              'Reference date ${_todayIso()}',
              style: const TextStyle(fontSize: 12, color: Colors.white70),
            ),
          ],
        ),
      ),
      body: fruitsAsync.when(
        data: (fruits) {
          // 1. Lọc nhóm Expiring soon: 0 <= daysLeft <= 7
          final expiringSoon = fruits
              .where((f) => f.daysLeft >= 0 && f.daysLeft <= 7)
              .toList()
            ..sort((a, b) => a.daysLeft.compareTo(b.daysLeft));
```

```
        // 2. Lọc nhóm Expired: daysLeft < 0
        final expired = fruits.where((f) => f.daysLeft < 0).toList()
          ..sort((a, b) => b.daysLeft.compareTo(a.daysLeft));

        return ListView(
          padding: const EdgeInsets.all(16.0),
          children: [
            // SECTION 1: EXPIRING SOON
            _buildSectionHeader(
              title: 'EXPIRING SOON',
              count: expiringSoon.length,
              badgeBg: Colors.orange.shade100,
              badgeFg: Colors.orange.shade900,
            ),
            const SizedBox(height: 8),
            if (expiringSoon.isEmpty)
              const Padding(
                padding: EdgeInsets.symmetric(vertical: 12),
                child: Text(
                  'No fruits expiring within 7 days.',
                  style: TextStyle(color: Colors.black54),
                ),
              )
            else
              ...expiringSoon.map((f) => _buildWarningTile(f, isExpired: false)),

            const SizedBox(height: 24),

            // SECTION 2: EXPIRED
            _buildSectionHeader(
              title: 'EXPIRED',
              count: expired.length,
              badgeBg: Colors.red.shade100,
              badgeFg: Colors.red.shade800,
            ),
            const SizedBox(height: 8),
            if (expired.isEmpty)
              const Padding(
                padding: EdgeInsets.symmetric(vertical: 12),
                child: Text(
                  'No expired fruits.',
                  style: TextStyle(color: Colors.black54),
                ),
              )
            else
              ...expired.map((f) => _buildWarningTile(f, isExpired: true)),
```

```
          ],
        );
      },
      error: (err, _) => Center(child: Text('Error: $err')),
      loading: () => const Center(child: CircularProgressIndicator()),
    ),
  );
}

Widget _buildSectionHeader({
  required String title,
  required int count,
  required Color badgeBg,
  required Color badgeFg,
}) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(
        title,
        style: TextStyle(
          fontSize: 13,
          fontWeight: FontWeight.bold,
          color: badgeFg,
          letterSpacing: 0.8,
        ),
      ),
      CircleAvatar(
        radius: 12,
        backgroundColor: badgeBg,
        child: Text(
          '$count',
          style: TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.bold,
            color: badgeFg,
          ),
        ),
      ),
    ],
  );
}

Widget _buildWarningTile(Fruit fruit, {required bool isExpired}) {
  final days = fruit.daysLeft;
  final originText = fruit.origin.trim().isEmpty ? 'N/A' : fruit.origin;
```

```
    final badgeText = isExpired
        ? '${days.abs()} day${days.abs() == 1 ? '' : 's'} ago'
        : '$days day${days == 1 ? '' : 's'} left';

    return Card(
      margin: const EdgeInsets.only(bottom: 8),
      elevation: 0.5,
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor:
              isExpired ? Colors.red.shade100 : Colors.green.shade100,
          child: Icon(
            Icons.eco,
            color: isExpired ? Colors.red.shade700 : Colors.green.shade800,
          ),
        ),
        title: Text(
          fruit.name,
          style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 15),
        ),
        subtitle: Text(
          'Exp ${fruit.expiredDate} · $originText',
          style: const TextStyle(fontSize: 12.5),
        ),
        trailing: Container(
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
          decoration: BoxDecoration(
            color: isExpired ? Colors.red.shade50 : Colors.orange.shade50,
            borderRadius: BorderRadius.circular(6),
          ),
          child: Text(
            badgeText,
            style: TextStyle(
              fontSize: 12,
              fontWeight: FontWeight.bold,
              color: isExpired ? Colors.red.shade800 : Colors.orange.shade900,
            ),
          ),
        ),
      ),
    );
  }
}
```

Bước 8: Cập Nhật `lib/main.dart` (Task 6 — Consistent Theming)

Giải thích

- Lưu ý nhỏ ở **Task 6** và trang **Illustrations**:
    - Văn bản Task 6 ghi *"color is orange"*, trong khi trang 3 Illustrations ghi *"Colors are dark green based on the fruit theme"*.
    - Trong `main.dart` dưới đây, mình thiết lập sẵn hằng số `primaryThemeColor = Color(0xFF1B5E20)` (Dark Green giống hệt 6 hình minh họa trang 3), và ghi chú ngay bên cạnh `Colors.orange` để nếu giám khảo chấm theo màu nào bạn cũng chỉ cần đổi đúng 1 từ!

Code `lib/main.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'ui/fruit_list_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: MyApp()));
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // Đổi thành Colors.orange.shade800 nếu muốn AppBar màu cam theo dòng chữ ở Task
6,
  // hoặc giữ Color(0xFF1B5E20) (Dark Green) giống hệt 6 hình Illustrations ở trang
3.
  static const Color primaryColor = Color(0xFF1B5E20);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Fruit Sales Management',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: primaryColor),
        useMaterial3: true,
        appBarTheme: const AppBarTheme(
          backgroundColor: primaryColor,
          foregroundColor: Colors.white,
        ),
        floatingActionButtonTheme: const FloatingActionButtonThemeData(
          backgroundColor: primaryColor,
          foregroundColor: Colors.white,
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: primaryColor,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(24),
            ),
          ),
        ),
```

```
      ),
      home: const FruitListScreen(),
    );
  }
}
```

5. Hướng Dẫn Kiểm Thử Với Bảng Sample Data (Trang 2)

1. Nếu muốn kiểm thử khớp y hệt kết quả mẫu ở trang 2 (`Reference date: 2026-11-08`), bạn có thể đổi ngày hệ thống của máy ảo Android về **`08/11/2026`** (hoặc tạm thay `DateTime.now()` trong `Fruit.computeDaysLeft` thành `DateTime(2026, 11, 8)` khi test, **nhớ trả lại `DateTime.now()` trước khi nộp**).
2. Nhập 10 dòng dữ liệu trong bảng Sample Data qua màn hình **Add Fruit**:
   - **Expired (`daysLeft < 0`) — Có 2 quả**:
     - `Buoi Da Xanh` (`2026-11-05` -> `3 days ago`)
     - `Nho Ninh Thuan` (`2026-11-02` -> `6 days ago`)
   - **Expiring soon (`0 <= daysLeft <= 7`) — Có 4 quả**:
     - `Chuoi Gia Huong` (`2026-11-08` -> `0 day left`)
     - `Xoai Cat Chu` (`2026-11-09` -> `1 day left`)
     - `Dau Tay Da Lat` (`2026-11-14` -> `6 days left`)
     - `Thanh Long` (`2026-11-15` -> `7 days left`)
   - **Fresh (`daysLeft > 7`) — Có 4 quả**:
     - `Bo Sap` (`2026-11-16`), `Cam Sanh` (`2026-11-20`), `Sau Rieng Ri6` (`2026-11-30`), `Mit Thai` (`2026-12-01`).
3. Trước khi đóng gói nộp bài, mở Terminal chạy:

```
bash
flutter clean
```